

Kayode Oladipo - assignment

SECURED SAFE VAULT API APPLICATION - REPORTS

SafeVault is a secure web application designed to manage sensitive data, including user credentials and financial records.

The AI was used to generate secure code for a web application, focusing on mitigating common vulnerabilities such as SQL injection and cross-site scripting (XSS). The codes were implemented and developed. Initially the application was not producing the expected outcomes so the AI was again used to debug and refine the codes. When the application worked fine, the desire to secure the application became indispensable to prevent malicious injections, eliminate SQL injection vulnerabilities and prevent XSS vulnerabilities through the injection of malicious scripts into user inputs. These requirements were achieved by the enforcing the use of parameterized queries to prevent SQL injection, the use of validation to validate user inputs to prevent malicious injections and securely handle user-provided data, such as login credentials or search inputs.

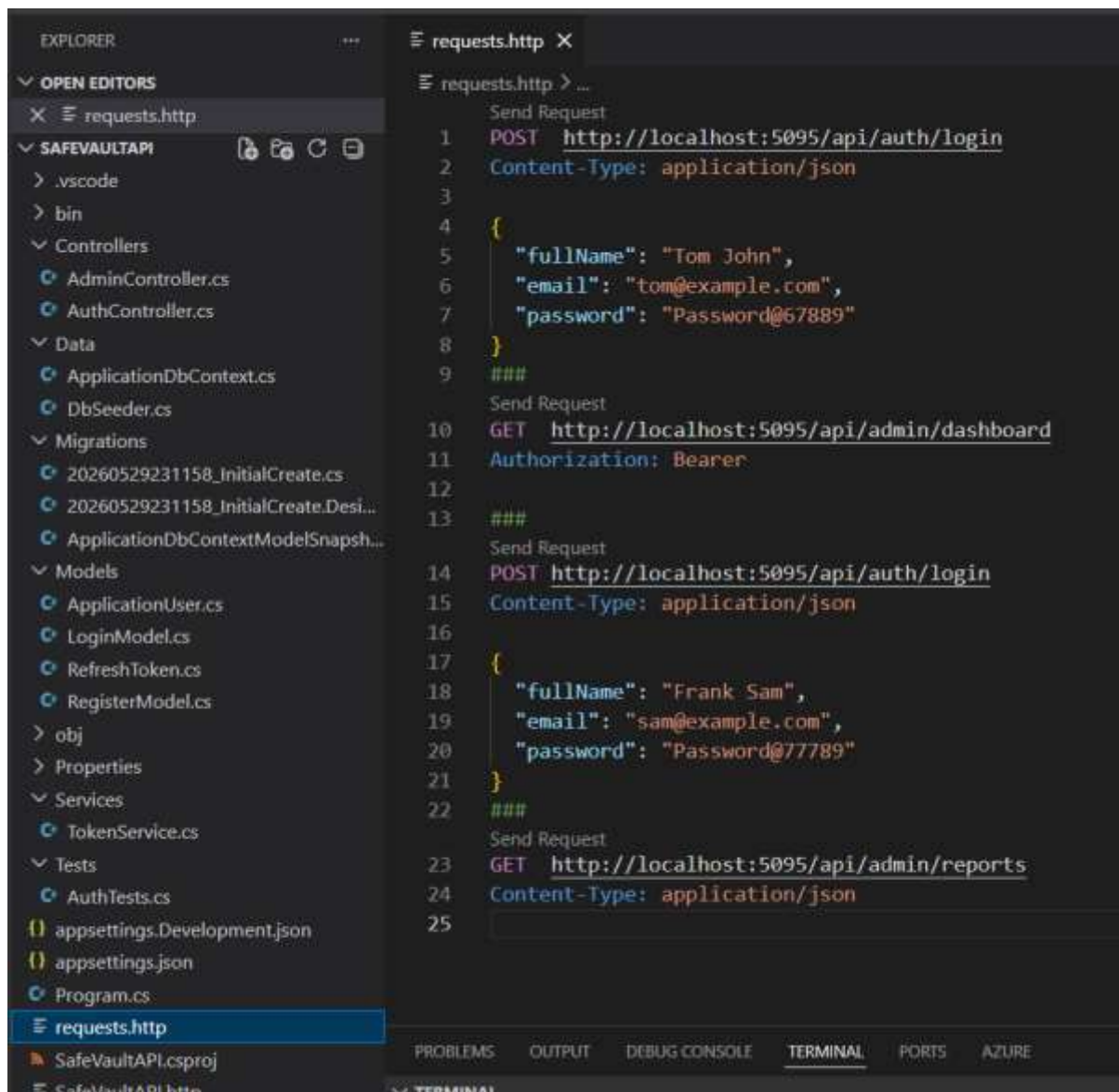
JWT validation is performed. JWT validation is like a checkpoint that confirms a token is trustworthy and hasn't been tampered with. Only tokens that pass this validation are allowed access. Another part of securing API endpoints with JWTs is using JWT middleware to automate token validation as a security gate for every request. Once set up, the middleware in ASP.NET Core checks each token's authenticity and expiration, blocking any requests with an invalid token.

Kayode Oladipo - assignment

The following shows the file structure of the SafeVaultAPI Application.

```
SAFEVAULTAPI/
├── .vscode/
├── bin/
├── obj/
├── Controllers/
│   ├── AdminController.cs
│   └── AuthController.cs
├── Data/
│   ├── ApplicationDbContext.cs
│   └── DbSeeder.cs
├── Migrations/
│   ├── 20260529231158_InitialCreate.cs
│   ├── 20260529231158_InitialCreate.Designer.cs
│   └── ApplicationDbContextModelSnapshot.cs
├── Models/
│   ├── ApplicationUser.cs
│   ├── LoginModel.cs
│   ├── RefreshToken.cs
│   └── RegisterModel.cs
├── Services/
│   └── TokenService.cs
├── Tests/
│   └── AuthTests.cs
├── Properties/
├── appsettings.json
├── appsettings.Development.json
├── Program.cs
├── requests.http
├── SafeVaultAPI.csproj
├── SafeVaultAPI.http
└── README / (not shown but usually here)
```

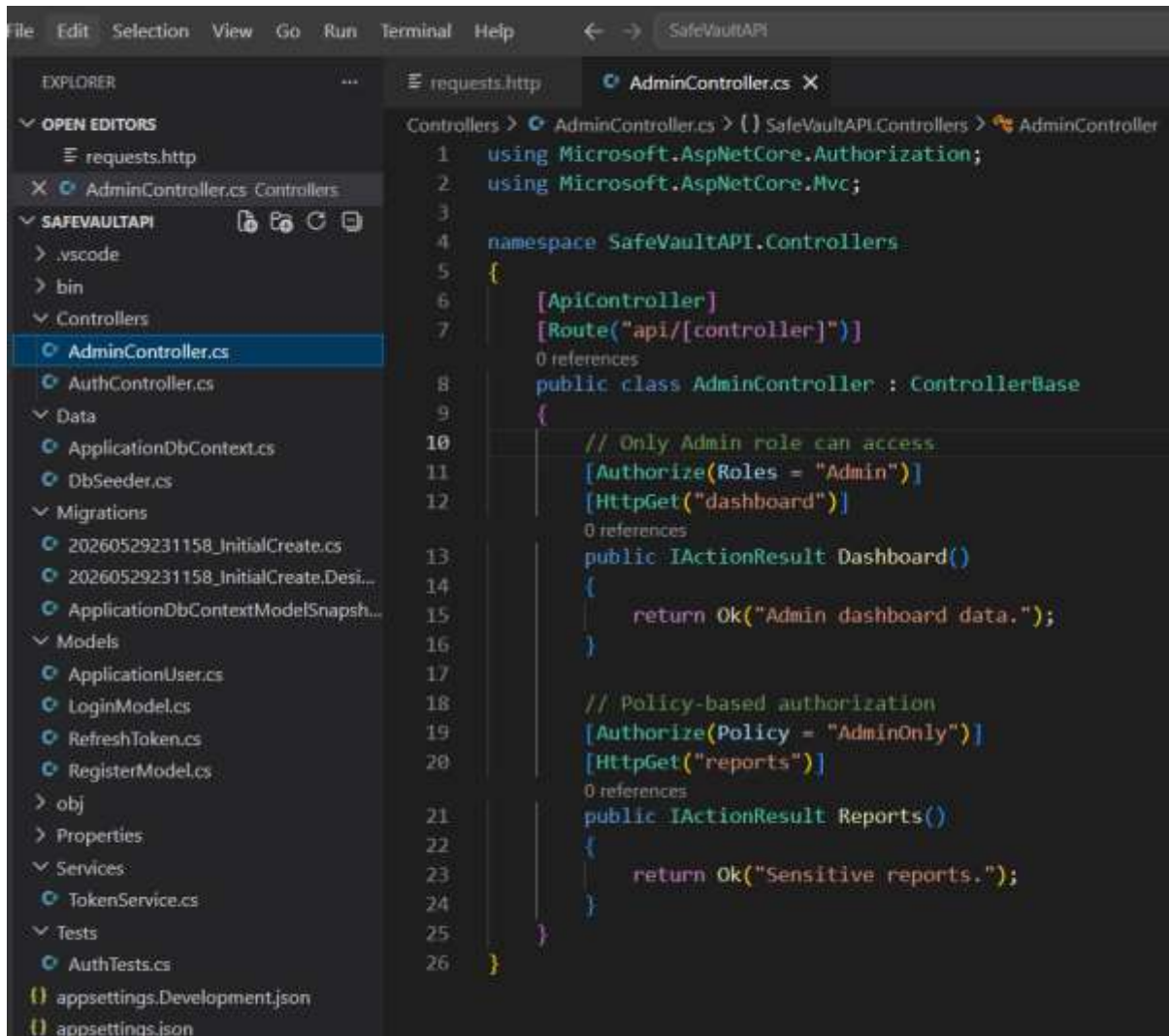
Kayode Oladipo - assignment



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar displays the project structure for 'SAFEVAULTAPI'. The 'requests.http' file is selected and open in the editor. The editor shows three HTTP requests, each preceded by a 'Send Request' comment. The first request is a POST to '/api/auth/login' with a JSON body. The second is a GET to '/api/admin/dashboard' with a Bearer token. The third is a GET to '/api/admin/reports'.

```
requests.http
1  Send Request
2  POST http://localhost:5095/api/auth/login
3
4  {
5    "fullName": "Tom John",
6    "email": "tom@example.com",
7    "password": "Password@67889"
8  }
9  ###
10 Send Request
11 GET http://localhost:5095/api/admin/dashboard
12
13 Authorization: Bearer
14
15 ###
16 Send Request
17 POST http://localhost:5095/api/auth/login
18
19 Content-Type: application/json
20
21 {
22   "fullName": "Frank Sam",
23   "email": "sam@example.com",
24   "password": "Password@77789"
25 }
```

Kayode Oladipo - assignment



The AuthController.cs (Security)

The program codes in the AuthController.cs file performs most of the security expected as shown below:

ASP.NET Identity Security

The biggest security feature comes from:

```
private readonly UserManager<ApplicationUser> _userManager;
```

ASP.NET Identity automatically provides several protections.

Password Hashing

During registration:

```
await _userManager.CreateAsync(user, model.Password);
```

Kayode Oladipo - assignment

The password is **not stored in plain text**.

ASP.NET Identity:

1. Generates a random salt.
2. Hashes the password.
3. Stores only the hash.

Example:

Instead of:

Password123!

Database stores something similar to:

```
AQAAAAEAACcQAAAAEM5...
```

Benefits:

- Database compromise does not reveal passwords directly.
- Rainbow table attacks are mitigated through salting.
- Modern hashing algorithms are used.

Password Verification

During login:

```
await _userManager.CheckPasswordAsync(user, model.Password);
```

The supplied password is hashed again and compared to the stored hash.

Benefits:

- Plain-text passwords never need to be retrieved.
- Secure comparison is performed by Identity.

2. Model Validation Protection

Both endpoints use:

```
if (!ModelState.IsValid)  
    return BadRequest(ModelState);
```

Assuming the models contain validation attributes:

Kayode Oladipo - assignment

```
[Required]
[EmailAddress]
```

or

```
[StringLength]
```

This protects against:

- ❖ Missing required fields (this is presence check)
- ❖ Invalid email formats
- ❖ Invalid data types
- ❖ Basic malformed requests

Duplicate Account Protection

Before registration:

```
var existingUser =
    await _userManager.FindByEmailAsync(model.Email);

if (existingUser != null)
    return BadRequest("Email already exists.");
```

The block of codes above Prevents:

- Multiple accounts with same email
- Identity confusion
- Account duplication

4. JWT Security Features

The JWT method contains multiple security mechanisms.

Digital Signature

```
var creds = new SigningCredentials(
    key, SecurityAlgorithms.HmacSha256);
```

Uses:

HMAC-SHA256 to sign the token. This prevents attackers from changing claims.

Kayode Oladipo - assignment

```
var key = new SymmetricSecurityKey(  
    Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));  
  
var creds = new SigningCredentials(  
    key,  
    SecurityAlgorithms.HmacSha256);  
  
var expires = DateTime.UtcNow.AddMinutes(  
    Convert.ToDouble(_configuration["Jwt:DurationInMinutes"]));
```

Without signing:

```
{  
  "role": "User"  
}
```

could be changed to:

```
{  
  "role": "Admin"  
}
```

With signing:

- Signature becomes invalid.
- Token is rejected.

Secret Key Protection

```
new SymmetricSecurityKey(Encoding.UTF8.GetBytes(  
    _configuration["Jwt:Key"]!));
```

The secret key is externalized.

Benefits:

- Key not hardcoded.
- Easier key rotation.
- Different environments can use different keys.

```
7  "Jwt": {  
8      "Key": "MyVeryStrongSecretKeyForJwtAuthentication2026!",  
9      "Issuer": "SafeVaultAPI",  
10     "Audience": "SafeVaultAPIUsers",  
11     "DurationInMinutes": 15  
12 },  
13
```

Kayode Oladipo - assignment

Expiration Protection

```
var expires = DateTime.UtcNow.AddMinutes(  
    Convert.ToDouble(_configuration["Jwt:DurationInMinutes"]));  
  
var token = new JwtSecurityToken(  
    issuer: _configuration["Jwt:Issuer"],  
    audience: _configuration["Jwt:Audience"],  
    claims: claims,  
    expires: expires,  
    signingCredentials: creds);
```

Benefits:

- Limits token lifetime.
- Reduces risk from stolen tokens.

Example:

Token valid:
12:00 -> 13:00

After expiration:

401 Unauthorized will be displayed. This is a protection mechanism.

Issuer Validation

issuer:
_configuration["Jwt:Issuer"]

Used to validate:

Who issued the token and hence prevents accepting tokens from other issuers.

```
var token = new JwtSecurityToken(  
    issuer: _configuration["Jwt:Issuer"],  
    audience: _configuration["Jwt:Audience"],  
    claims: claims,  
    expires: expires,  
    signingCredentials: creds);
```

Audience Validation

Kayode Oladipo - assignment

```
audience:  
_configuration["Jwt:Audience"]
```

Defines:

Who is allowed to use the token? This prevents token reuse across applications.

Example:

Token for:

SafeVaultAPI should not work in: PayrollAPI

5. Role-Based Authorization Support

Roles are retrieved:

```
var roles = await _userManager.GetRolesAsync(user);
```

and added as claims:

```
claims.Add(new Claim(ClaimTypes.Role, role));
```

This enables:

```
[Authorize(Roles="Admin")]
```

Example:

```
{  
  "role": "Admin"  
}
```

can later be used for access control.

6. Logging and Auditing

Successful registration:

```
_logger.LogInformation(...)
```

Failed login:

```
_logger.LogWarning(...)
```

Provides:

Kayode Oladipo - assignment

- ❖ Audit trail
- ❖ Security monitoring
- ❖ Attack investigation

Example log:

```
Login failed. Invalid password:  
john@email.com
```

Useful for:

- ❖ Brute force detection
- ❖ Account abuse detection
- ❖ Incident response

```
// =====  
// Login User  
// =====  
[HttpPost("login")]  
0 references  
public async Task<IActionResult> Login(LoginModel model)  
{  
    if (!ModelState.IsValid)  
        return BadRequest(ModelState);  
  
    var user =  
        await _userManager.FindByEmailAsync(model.Email);  
  
    if (user == null)  
    {  
        _logger.LogWarning(  
            $"Login failed. User not found: {model.Email}");  
  
        return Unauthorized("Invalid credentials.");  
    }  
  
    var passwordValid =  
        await _userManager.CheckPasswordAsync(  
            user,  
            model.Password);  
  
    if (!passwordValid)  
    {  
        _logger.LogWarning(  
            $"Login failed. Invalid password: {model.Email}");  
  
        return Unauthorized("Invalid credentials.");  
    }  
}
```

4. Login Enumeration Through Logs

Logs reveal:

User not found

vs

Invalid password

Although the API response is identical:

Unauthorized("Invalid credentials.")

Kayode Oladipo - assignment

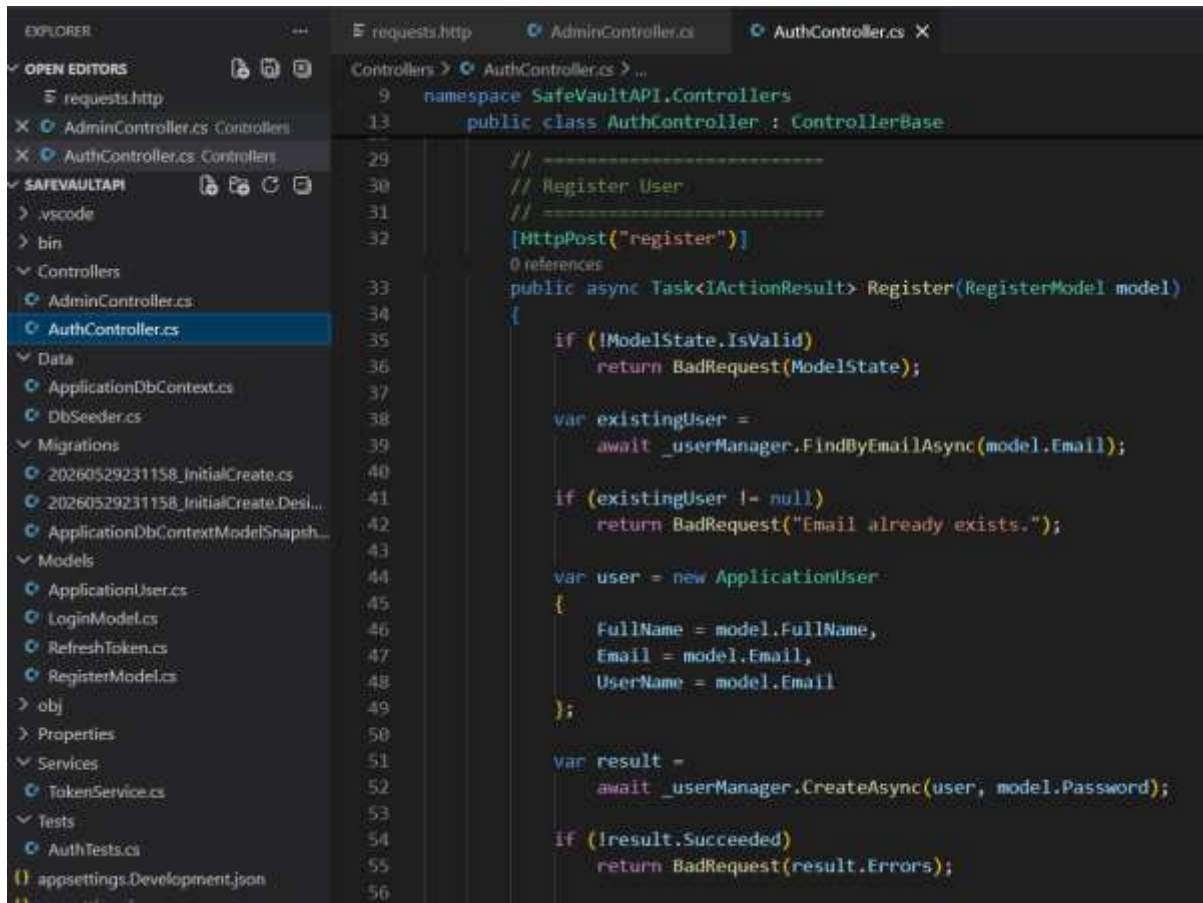
the logs reveal account existence.

This may be acceptable for internal logs but should be protected carefully.

FULL IMPLEMENTATION OF THE SECURITY WITHIN AuthController.cs

The image is a screenshot of the Visual Studio IDE. On the left, the 'EXPLORER' pane shows the project structure. Under 'SAFEVAULTAPI', there are folders for '.vscode', 'bin', 'Controllers', 'Data', 'Migrations', 'Models', 'obj', 'Properties', 'Services', 'Tests', and files for 'appsettings.Development.json' and 'appsettings.json'. The 'Controllers' folder is expanded, showing 'AdminController.cs' and 'AuthController.cs'. 'AuthController.cs' is selected. The main editor pane shows the code for 'AuthController.cs'. The code includes using statements for Microsoft.AspNetCore.Identity, Microsoft.AspNetCore.Mvc, Microsoft.IdentityModel.Tokens, SafeVaultAPI.Models, System.IdentityModel.Tokens.Jwt, System.Security.Claims, and System.Text. It defines a namespace 'SafeVaultAPI.Controllers' and a class 'AuthController' that inherits from 'ControllerBase'. The class has three private readonly fields: 'userManager' of type 'UserManager<ApplicationUser>', 'configuration' of type 'IConfiguration', and 'logger' of type 'ILogger<AuthController>'. The constructor 'AuthController' takes these three parameters and assigns them to the fields. The class is decorated with '[ApiController]' and '[Route(\"api/[controller]\")]'. The code is numbered from 1 to 27.

Kayode Oladipo - assignment



This screenshot shows the Visual Studio IDE with the 'AuthController.cs' file open. The left sidebar displays the 'EXPLORER' view with a project structure for 'SAFEVAULTAPI'. The main editor area shows the 'Register' method in the 'AuthController' class. The code includes a namespace declaration, a class declaration, a POST attribute, and a method body that validates the model, checks for existing users, creates a new user, and returns a response.

```
namespace SafeVaultAPI.Controllers
{
    public class AuthController : ControllerBase
    {
        // =====
        // Register User
        // =====
        [HttpPost("register")]
        public async Task<IActionResult> Register(RegisterModel model)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

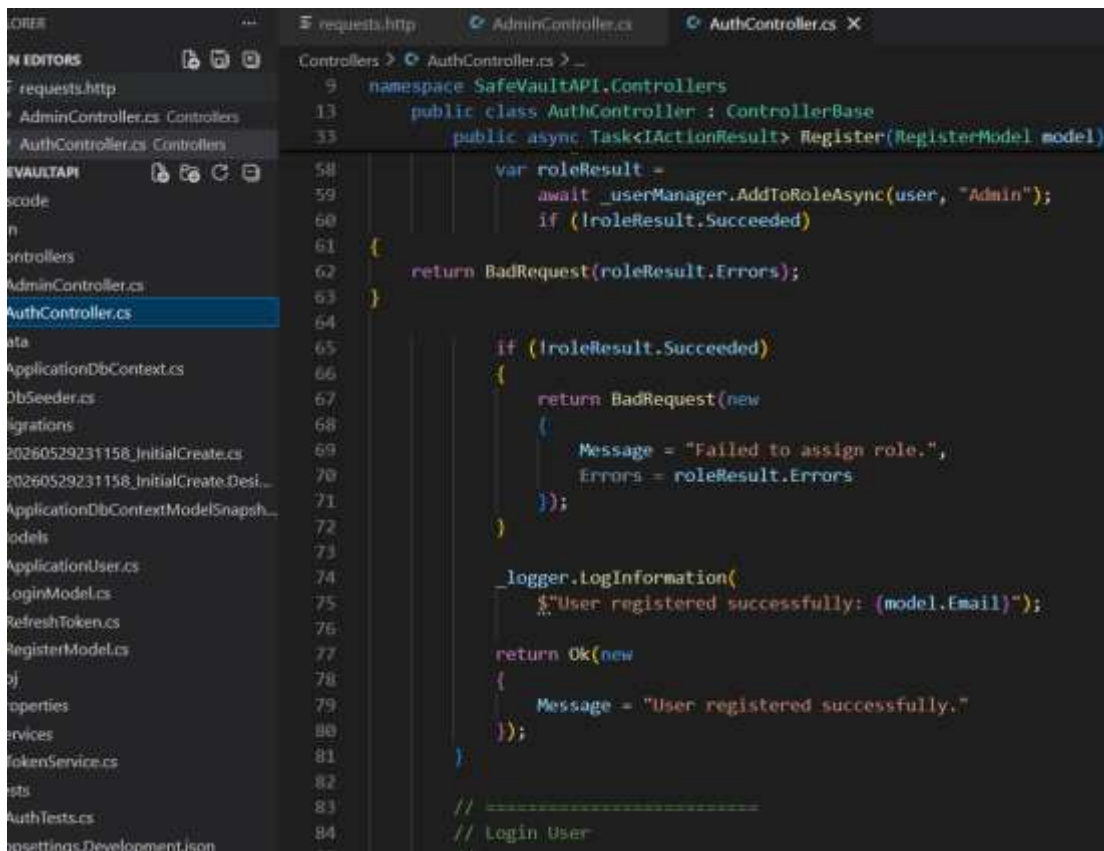
            var existingUser =
                await _userManager.FindByEmailAsync(model.Email);

            if (existingUser != null)
                return BadRequest("Email already exists.");

            var user = new ApplicationUser
            {
                FullName = model.FullName,
                Email = model.Email,
                UserName = model.Email
            };

            var result =
                await _userManager.CreateAsync(user, model.Password);

            if (!result.Succeeded)
                return BadRequest(result.Errors);
        }
    }
}
```



This screenshot shows the Visual Studio IDE with the 'AuthController.cs' file open. The left sidebar displays the 'EXPLORER' view with a project structure for 'SAFEVAULTAPI'. The main editor area shows the 'Register' method in the 'AuthController' class. The code includes a namespace declaration, a class declaration, a POST attribute, and a method body that validates the model, checks for existing users, creates a new user, assigns a role, and returns a response.

```
namespace SafeVaultAPI.Controllers
{
    public class AuthController : ControllerBase
    {
        public async Task<IActionResult> Register(RegisterModel model)
        {
            var roleResult =
                await _userManager.AddToRoleAsync(user, "Admin");

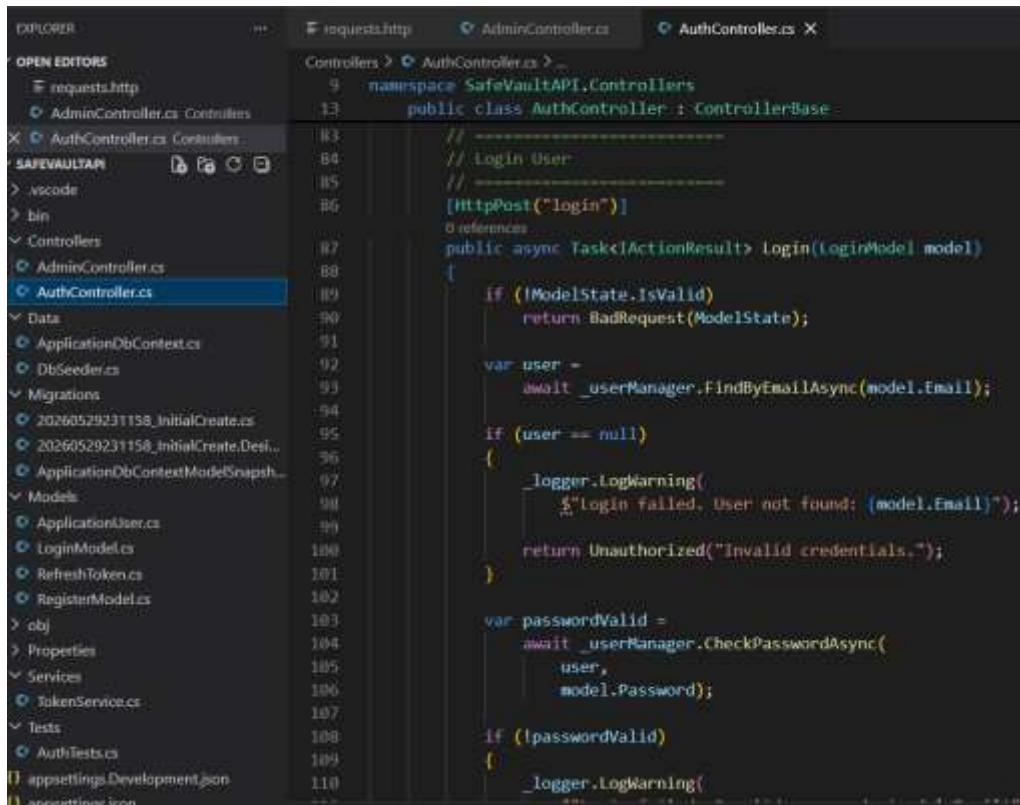
            if (!roleResult.Succeeded)
            {
                return BadRequest(roleResult.Errors);
            }

            if (!roleResult.Succeeded)
            {
                return BadRequest(new
                {
                    Message = "Failed to assign role.",
                    Errors = roleResult.Errors
                });
            }

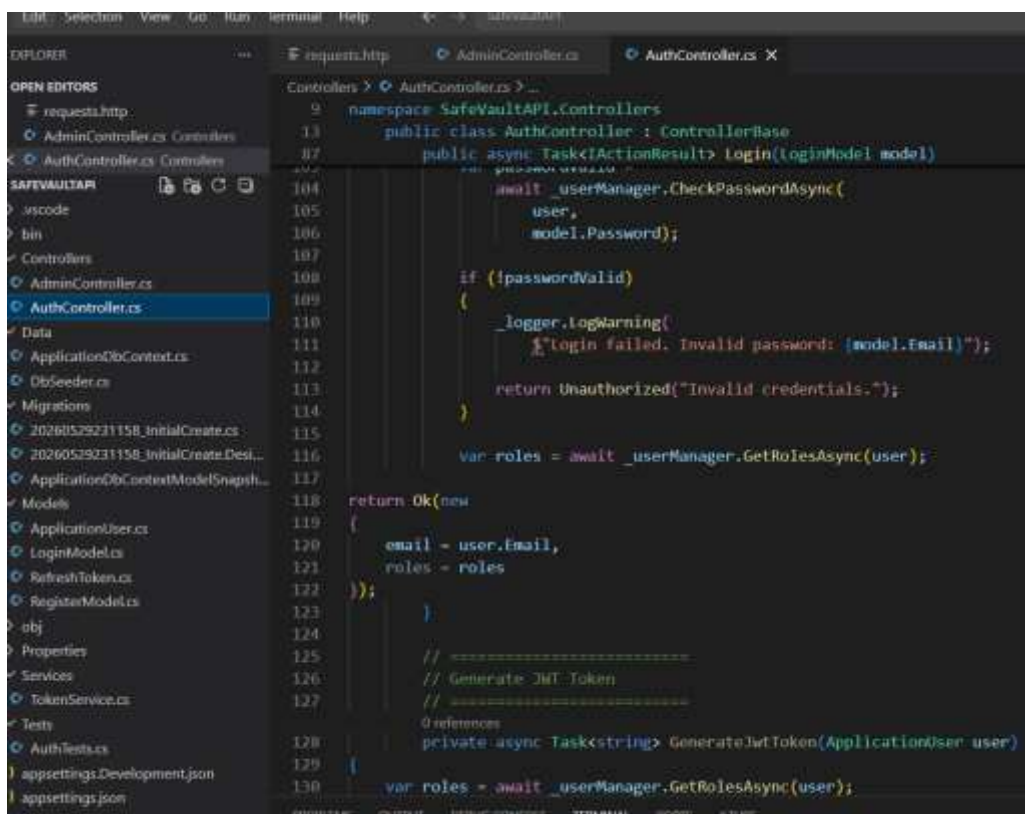
            _logger.LogInformation(
                $"User registered successfully: {model.Email}");

            return Ok(new
            {
                Message = "User registered successfully."
            });
        }
    }
}
```

Kayode Oladipo - assignment

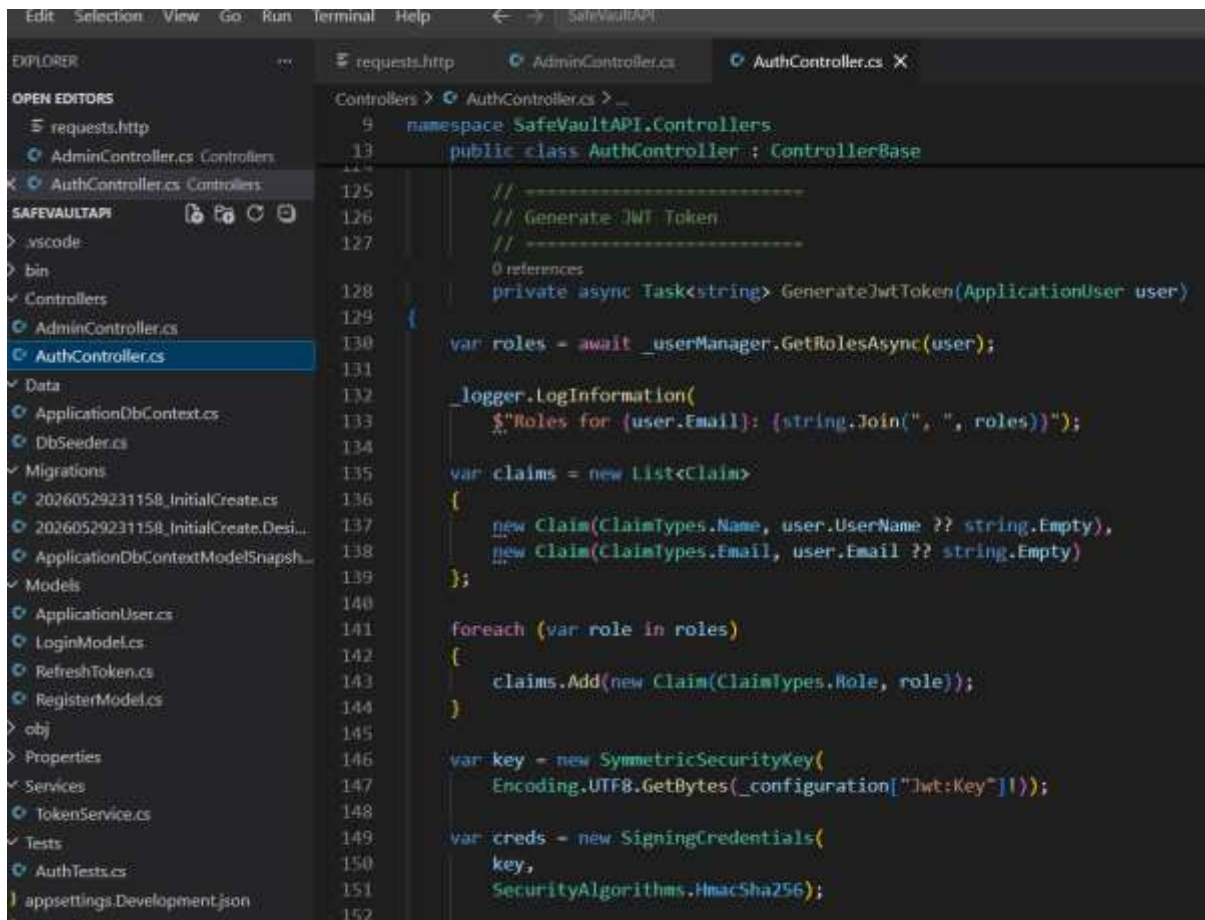


```
9 namespace SafeVaultAPI.Controllers
13 public class AuthController : ControllerBase
14 {
15     // Login User
16     // =====
17     [HttpPost("login")]
18     [ValidateAntiForgeryToken]
19     public async Task<ActionResult> login(LoginModel model)
20     {
21         if (!ModelState.IsValid)
22             return BadRequest(ModelState);
23
24         var user =
25             await _userManager.FindByEmailAsync(model.Email);
26
27         if (user == null)
28         {
29             _logger.LogWarning(
30                 $"login failed. User not found: {model.Email}");
31
32             return Unauthorized("Invalid credentials.");
33         }
34
35         var passwordValid =
36             await _userManager.CheckPasswordAsync(
37                 user,
38                 model.Password);
39
40         if (!passwordValid)
41         {
42             _logger.LogWarning(
43                 $"login failed. Invalid password: {model.Email}");
44
45             return Unauthorized("Invalid credentials.");
46         }
47
48         var roles = await _userManager.GetRolesAsync(user);
49
50         return Ok(new
51         {
52             email = user.Email,
53             roles = roles
54         });
55     }
56
57     // =====
58     // Generate JWT Token
59     // =====
60     [HttpPost("generate-token")]
61     [ValidateAntiForgeryToken]
62     public async Task<string> GenerateJwtToken(ApplicationUser user)
63     {
64         var roles = await _userManager.GetRolesAsync(user);
65
66         return Ok(new
67         {
68             email = user.Email,
69             roles = roles
70         });
71     }
72 }
```

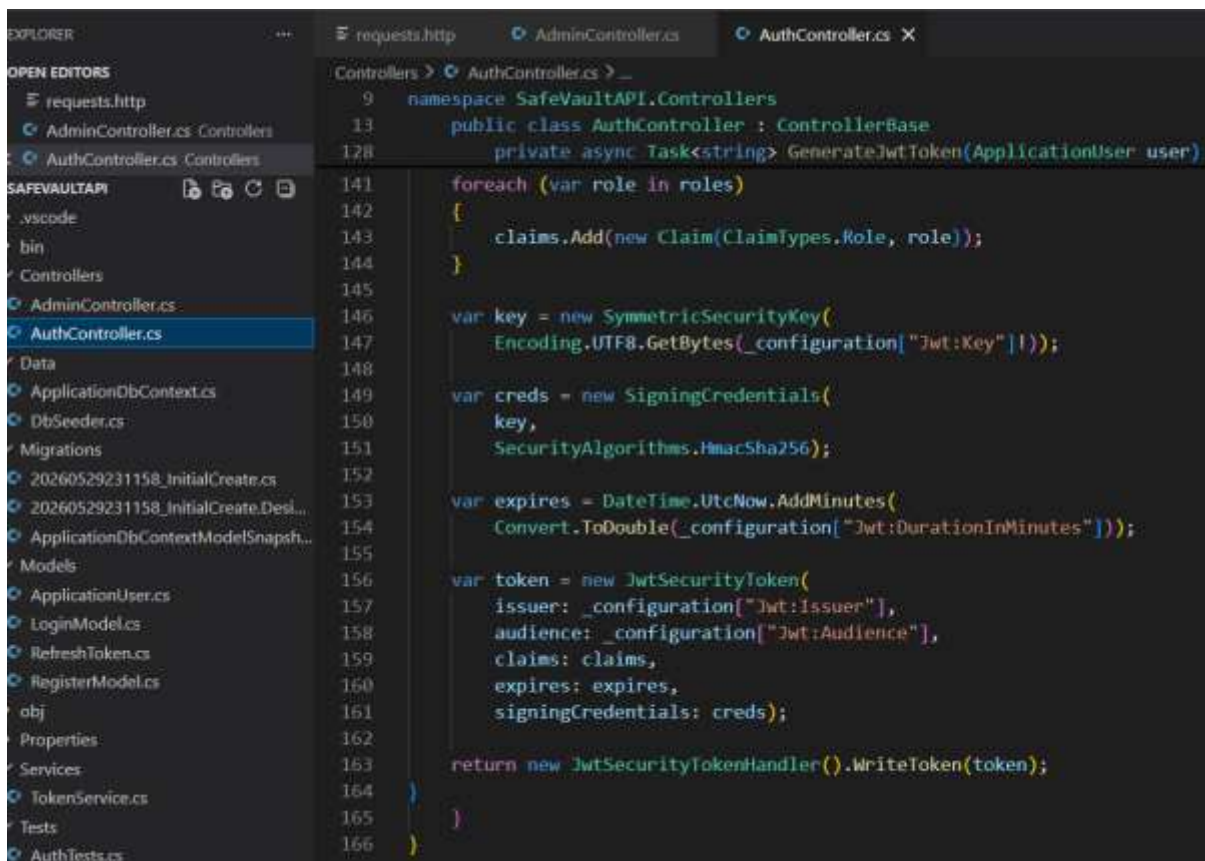


```
9 namespace SafeVaultAPI.Controllers
13 public class AuthController : ControllerBase
14 {
15     // Login User
16     // =====
17     [HttpPost("login")]
18     [ValidateAntiForgeryToken]
19     public async Task<ActionResult> login(LoginModel model)
20     {
21         if (!ModelState.IsValid)
22             return BadRequest(ModelState);
23
24         var user =
25             await _userManager.FindByEmailAsync(model.Email);
26
27         if (user == null)
28         {
29             _logger.LogWarning(
30                 $"login failed. User not found: {model.Email}");
31
32             return Unauthorized("Invalid credentials.");
33         }
34
35         var passwordValid =
36             await _userManager.CheckPasswordAsync(
37                 user,
38                 model.Password);
39
40         if (!passwordValid)
41         {
42             _logger.LogWarning(
43                 $"login failed. Invalid password: {model.Email}");
44
45             return Unauthorized("Invalid credentials.");
46         }
47
48         var roles = await _userManager.GetRolesAsync(user);
49
50         return Ok(new
51         {
52             email = user.Email,
53             roles = roles
54         });
55     }
56
57     // =====
58     // Generate JWT Token
59     // =====
60     [HttpPost("generate-token")]
61     [ValidateAntiForgeryToken]
62     public async Task<string> GenerateJwtToken(ApplicationUser user)
63     {
64         var roles = await _userManager.GetRolesAsync(user);
65
66         return Ok(new
67         {
68             email = user.Email,
69             roles = roles
70         });
71     }
72 }
```


Kayode Oladipo - assignment



```
9 namespace SafeVaultAPI.Controllers
13 public class AuthController : ControllerBase
14 {
15     // =====
16     // Generate JWT Token
17     // =====
18     0 references
19     private async Task<string> GenerateJwtToken(ApplicationUser user)
20     {
21         var roles = await _userManager.GetRolesAsync(user);
22
23         _logger.LogInformation(
24             $"Roles for {user.Email}: {string.Join(", ", roles)}");
25
26         var claims = new List<Claim>
27         {
28             new Claim(ClaimTypes.Name, user.UserName ?? string.Empty),
29             new Claim(ClaimTypes.Email, user.Email ?? string.Empty)
30         };
31
32         foreach (var role in roles)
33         {
34             claims.Add(new Claim(ClaimTypes.Role, role));
35         }
36
37         var key = new SymmetricSecurityKey(
38             Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));
39
40         var creds = new SigningCredentials(
41             key,
42             SecurityAlgorithms.HmacSha256);
43     }
44 }
```

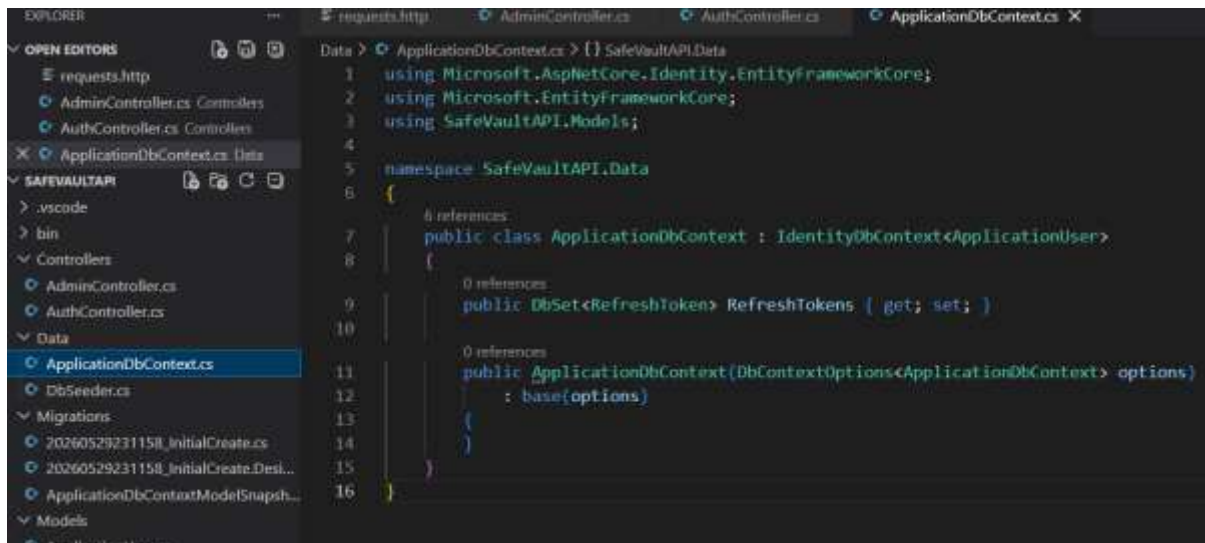


```
141     foreach (var role in roles)
142     {
143         claims.Add(new Claim(ClaimTypes.Role, role));
144     }
145
146     var key = new SymmetricSecurityKey(
147         Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));
148
149     var creds = new SigningCredentials(
150         key,
151         SecurityAlgorithms.HmacSha256);
152
153     var expires = DateTime.UtcNow.AddMinutes(
154         Convert.ToDouble(_configuration["Jwt:DurationInMinutes"]));
155
156     var token = new JwtSecurityToken(
157         issuer: _configuration["Jwt:Issuer"],
158         audience: _configuration["Jwt:Audience"],
159         claims: claims,
160         expires: expires,
161         signingCredentials: creds);
162
163     return new JwtSecurityTokenHandler().WriteToken(token);
164 }
165
166 }
```

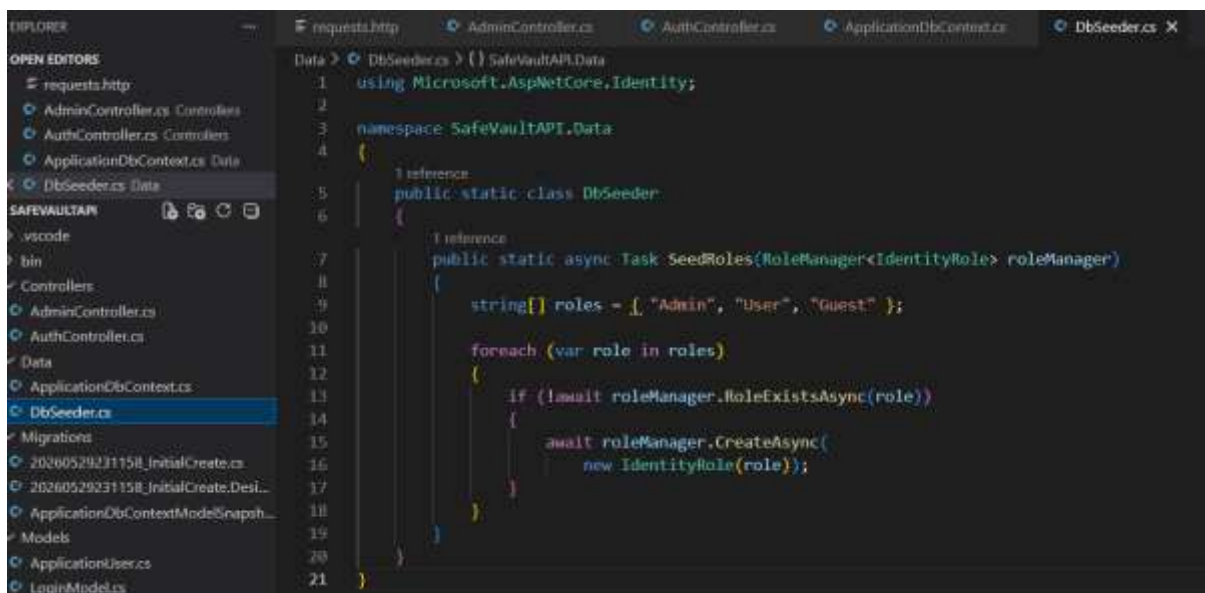
Kayode Oladipo - assignment

APPLICATION DB CONTEXT

ApplicationDbContext : is the class that acts as the application's connection to the database.



```
1 using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
2 using Microsoft.EntityFrameworkCore;
3 using SafeVaultAPI.Models;
4
5 namespace SafeVaultAPI.Data
6 {
7     6 references
8     public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
9     {
10         0 references
11         public DbSet<RefreshToken> RefreshTokens { get; set; }
12
13         0 references
14         public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
15             : base(options)
16     }
```

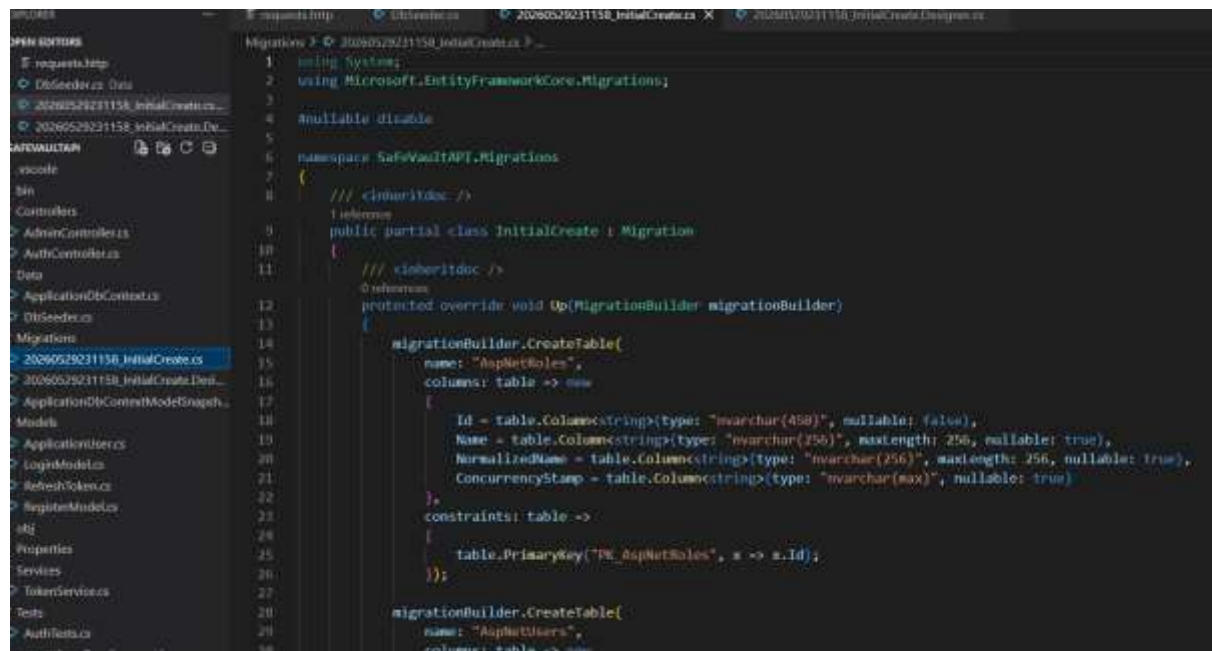


```
1 using Microsoft.AspNetCore.Identity;
2
3 namespace SafeVaultAPI.Data
4 {
5     1 reference
6     public static class DbSeeder
7     {
8         1 reference
9         public static async Task SeedRoles(RoleManager<IdentityRole> roleManager)
10         {
11             string[] roles = { "Admin", "User", "Guest" };
12
13             foreach (var role in roles)
14             {
15                 if (!await roleManager.RoleExistsAsync(role))
16                 {
17                     await roleManager.CreateAsync(
18                         new IdentityRole(role));
19                 }
20             }
21 }
```


Kayode Oladipo - assignment

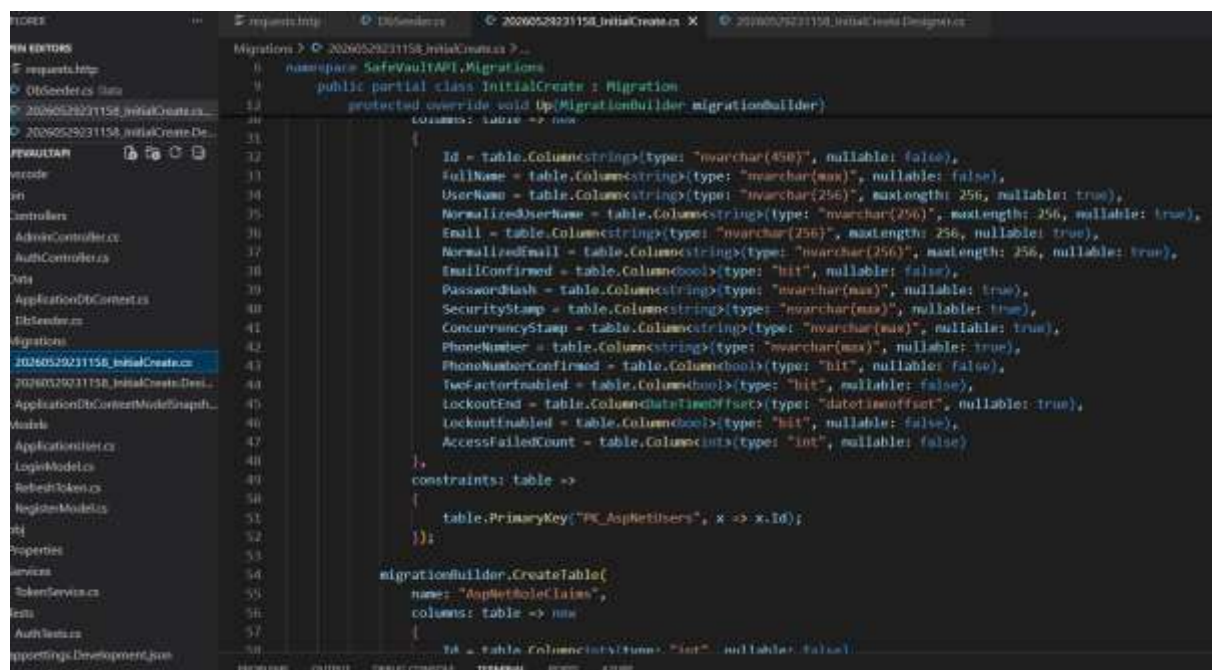
MIGRATIONS

A migration in Entity Framework Core is a mechanism for keeping the database schema synchronized with your C# model classes.



```
1 using System;
2 using Microsoft.EntityFrameworkCore.Migrations;
3
4 #nullable disable
5
6 namespace SafeVaultAPI.Migrations
7 {
8     /// <inheritdoc />
9     [inheritdoc]
10     public partial class InitialCreate : Migration
11     {
12         /// <inheritdoc />
13         [inheritdoc]
14         protected override void Up(MigrationBuilder migrationBuilder)
15         {
16             migrationBuilder.CreateTable(
17                 name: "AspNetRoles",
18                 columns: table => new
19                 {
20                     Id = table.Column<string>(type: "nvarchar(450)", nullable: false),
21                     Name = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
22                     NormalizedName = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
23                     ConcurrencyStamp = table.Column<string>(type: "nvarchar(max)", nullable: true)
24                 },
25                 constraints: table =>
26                 {
27                     table.PrimaryKey("PK_AspNetRoles", x => x.Id);
28                 });
29             migrationBuilder.CreateTable(
30                 name: "AspNetUsers",
31                 columns: table => new
```

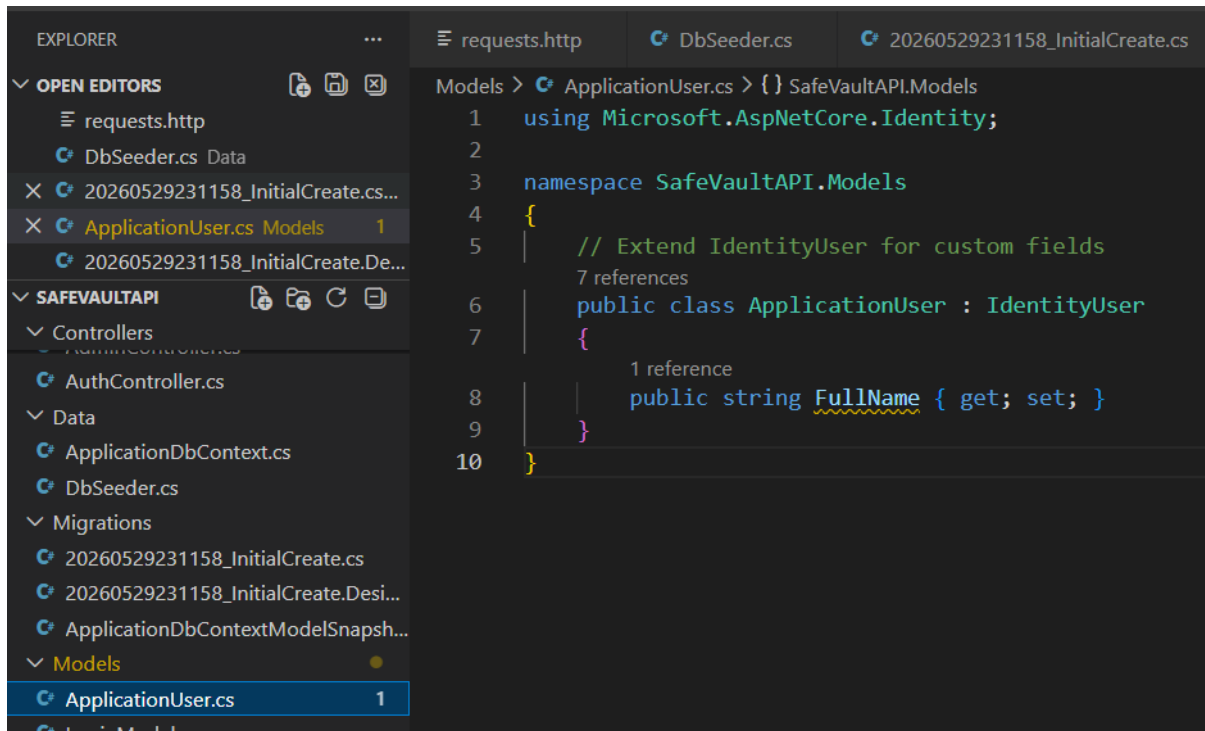
Creating the database entities and attributes based on the C# OOP models created.



```
32         {
33             Id = table.Column<string>(type: "nvarchar(450)", nullable: false),
34             FullName = table.Column<string>(type: "nvarchar(max)", nullable: false),
35             Username = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
36             NormalizedUsername = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
37             Email = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
38             NormalizedEmail = table.Column<string>(type: "nvarchar(256)", maxLength: 256, nullable: true),
39             EmailConfirmed = table.Column<bool>(type: "bit", nullable: false),
40             PasswordHash = table.Column<string>(type: "nvarchar(max)", nullable: true),
41             SecurityStamp = table.Column<string>(type: "nvarchar(max)", nullable: true),
42             ConcurrencyStamp = table.Column<string>(type: "nvarchar(max)", nullable: true),
43             PhoneNumber = table.Column<string>(type: "nvarchar(max)", nullable: true),
44             PhoneNumberConfirmed = table.Column<bool>(type: "bit", nullable: false),
45             TwoFactorEnabled = table.Column<bool>(type: "bit", nullable: false),
46             LockoutEnd = table.Column<DateTimeOffset>(type: "datetimeoffset", nullable: true),
47             LockoutEnabled = table.Column<bool>(type: "bit", nullable: false),
48             AccessFailedCount = table.Column<int>(type: "int", nullable: false)
49         },
50         constraints: table =>
51         {
52             table.PrimaryKey("PK_AspNetUsers", x => x.Id);
53         });
54             migrationBuilder.CreateTable(
55                 name: "AspNetRoleClaims",
56                 columns: table => new
57                 {
58                     Id = table.Column<int>(type: "int", nullable: false),
59                     RoleId = table.Column<string>(type: "nvarchar(450)", nullable: false),
60                     ClaimId = table.Column<int>(type: "int", nullable: false),
61                     ClaimValue = table.Column<string>(type: "nvarchar(max)", nullable: true)
62                 },
63                 constraints: table =>
64                 {
65                     table.PrimaryKey("PK_AspNetRoleClaims", x => x.Id);
66                     table.ForeignKey("FK_AspNetRoleClaims_AspNetRoles_RoleId", x => x.RoleId, "AspNetRoles", y => y.Id);
67                     table.ForeignKey("FK_AspNetRoleClaims_AspNetUsers_ClaimId", x => x.ClaimId, "AspNetUsers", y => y.Id);
68                 });
69             migrationBuilder.CreateTable(
70                 name: "AspNetUserClaims",
71                 columns: table => new
72                 {
73                     Id = table.Column<int>(type: "int", nullable: false),
74                     UserId = table.Column<string>(type: "nvarchar(450)", nullable: false),
75                     ClaimId = table.Column<int>(type: "int", nullable: false),
76                     ClaimValue = table.Column<string>(type: "nvarchar(max)", nullable: true)
77                 },
78                 constraints: table =>
79                 {
80                     table.PrimaryKey("PK_AspNetUserClaims", x => x.Id);
81                     table.ForeignKey("FK_AspNetUserClaims_AspNetUsers_UserId", x => x.UserId, "AspNetUsers", y => y.Id);
82                     table.ForeignKey("FK_AspNetUserClaims_AspNetRoles_ClaimId", x => x.ClaimId, "AspNetRoles", y => y.Id);
83                 });
84             migrationBuilder.CreateTable(
85                 name: "AspNetUserLogins",
86                 columns: table => new
87                 {
88                     LoginProvider = table.Column<string>(type: "nvarchar(450)", nullable: false),
89                     ProviderKey = table.Column<string>(type: "nvarchar(450)", nullable: false),
90                     ProviderDisplayName = table.Column<string>(type: "nvarchar(max)", nullable: true),
91                     UserId = table.Column<string>(type: "nvarchar(450)", nullable: false)
92                 },
93                 constraints: table =>
94                 {
95                     table.PrimaryKey("PK_AspNetUserLogins", x => x.LoginProvider);
96                     table.ForeignKey("FK_AspNetUserLogins_AspNetUsers_UserId", x => x.UserId, "AspNetUsers", y => y.Id);
97                     table.ForeignKey("FK_AspNetUserLogins_AspNetRoles_ProviderKey", x => x.ProviderKey, "AspNetRoles", y => y.Id);
98                 });
99             migrationBuilder.CreateTable(
100                 name: "AspNetUserTokens",
101                 columns: table => new
102                 {
103                     UserId = table.Column<string>(type: "nvarchar(450)", nullable: false),
104                     LoginProvider = table.Column<string>(type: "nvarchar(450)", nullable: false),
105                     Name = table.Column<string>(type: "nvarchar(450)", nullable: false),
106                     Value = table.Column<string>(type: "nvarchar(max)", nullable: true)
107                 },
108                 constraints: table =>
109                 {
110                     table.PrimaryKey("PK_AspNetUserTokens", x => x.UserId);
111                     table.ForeignKey("FK_AspNetUserTokens_AspNetUsers_UserId", x => x.UserId, "AspNetUsers", y => y.Id);
112                     table.ForeignKey("FK_AspNetUserTokens_AspNetRoles_LoginProvider", x => x.LoginProvider, "AspNetRoles", y => y.Id);
113                 });
114         }
115     }
116 }
```

Kayode Oladipo - assignment

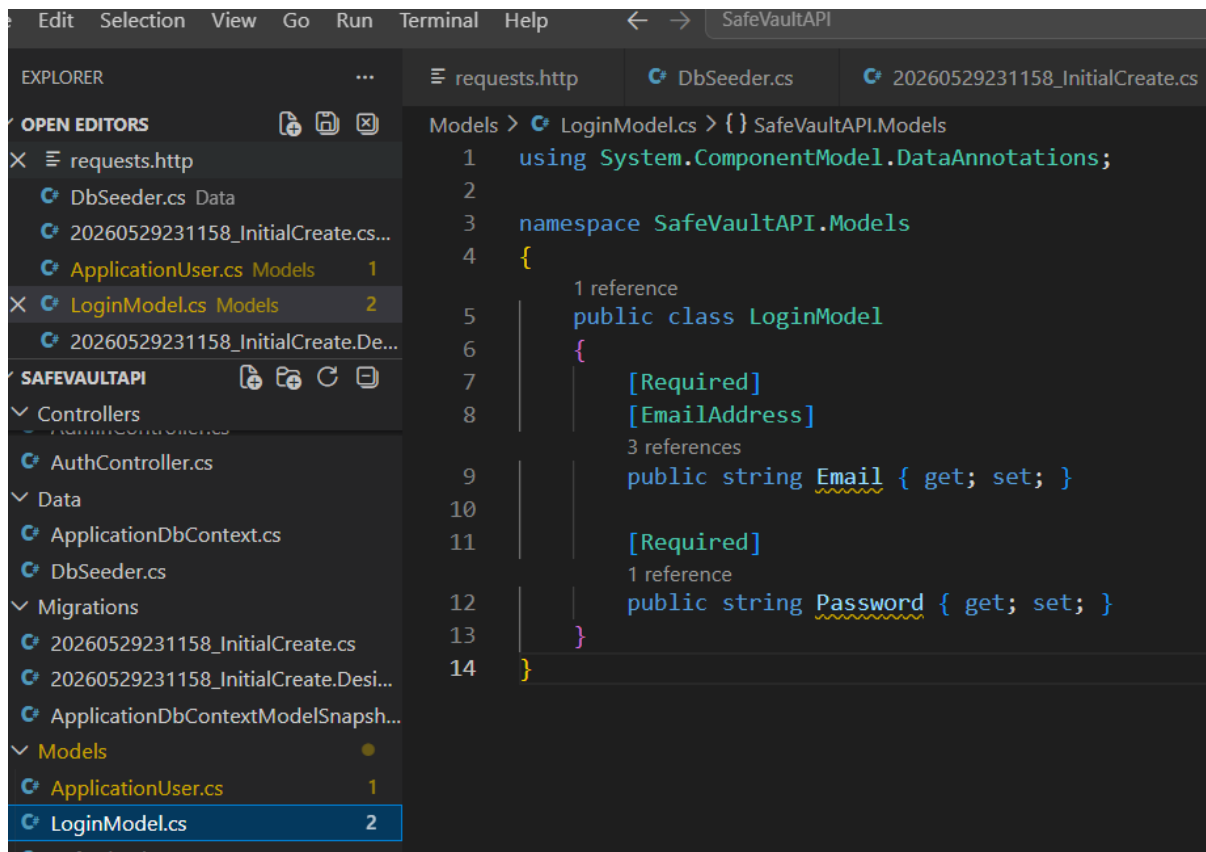
Here the IdentityUser class inherits from the ApplicationUser.



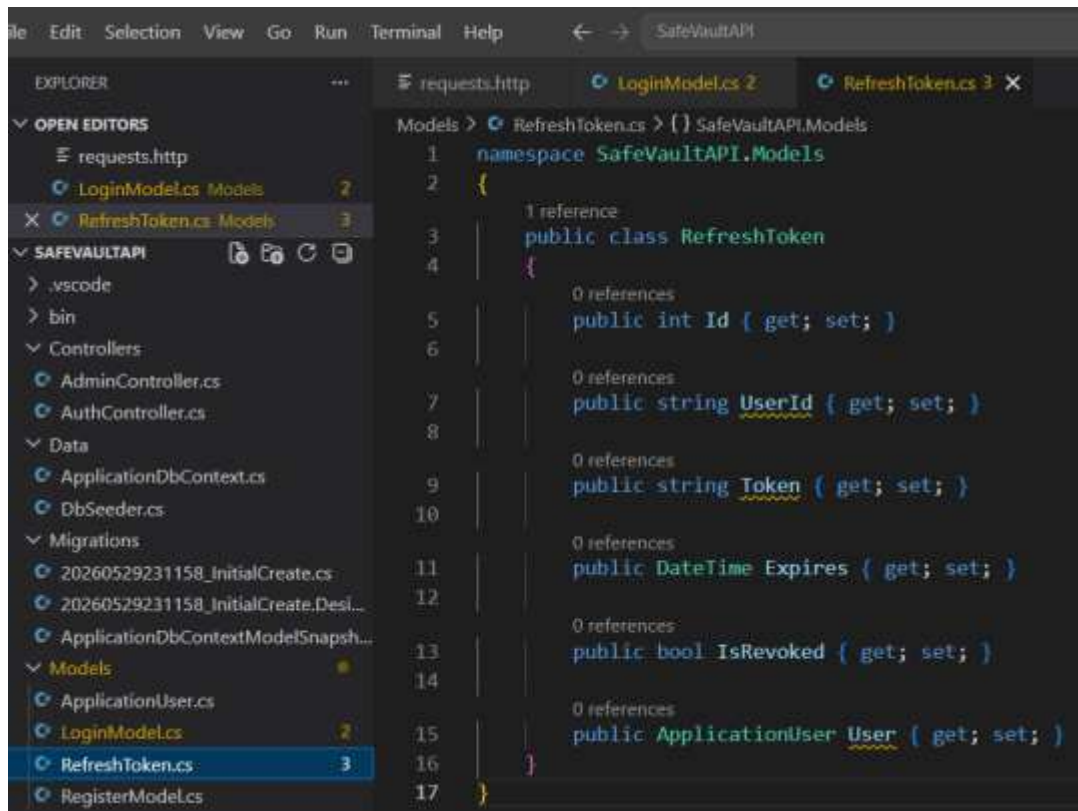
```
1  using Microsoft.AspNetCore.Identity;
2
3  namespace SafeVaultAPI.Models
4  {
5      // Extend IdentityUser for custom fields
6      // 7 references
7      public class ApplicationUser : IdentityUser
8      {
9          // 1 reference
10         public string FullName { get; set; }
11     }
12 }
```

Here is the Login Model used to generate the database components for emails and the password.

Kayode Oladipo - assignment

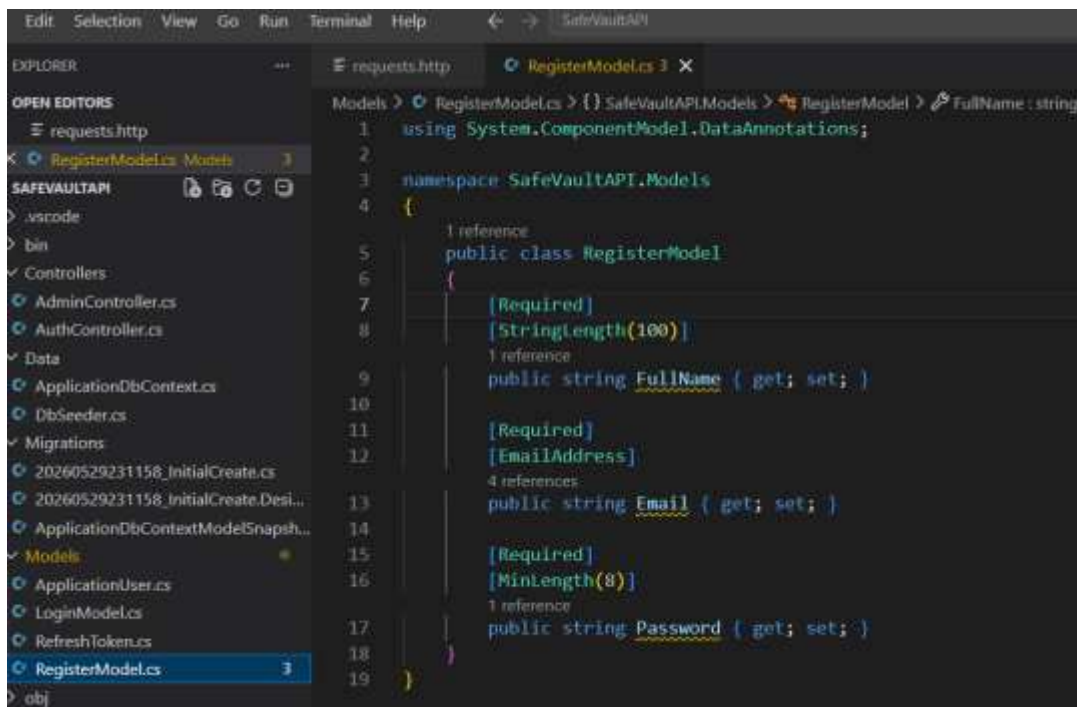


This is the model class that creates the token



Kayode Oladipo - assignment

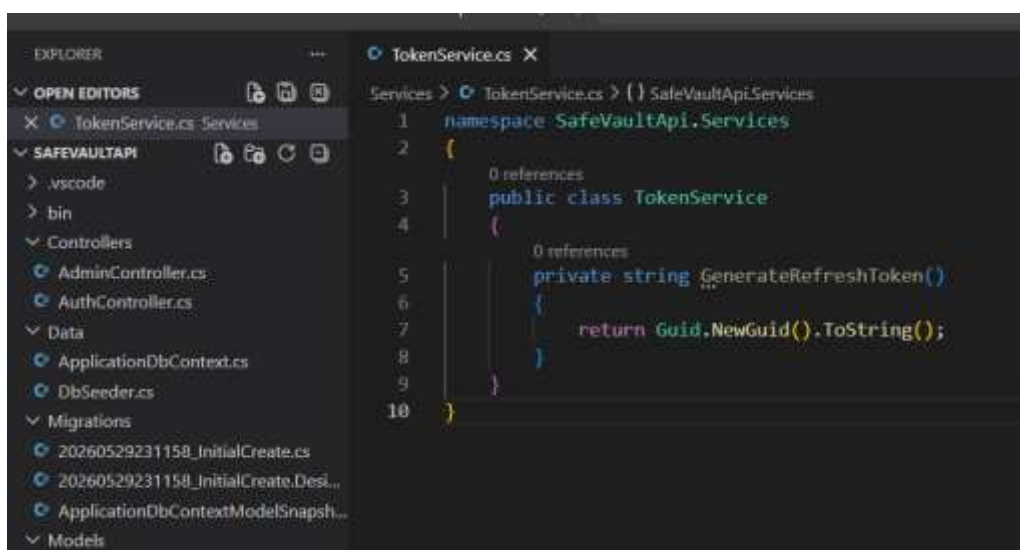
Here is the new user registration model; each attribute is validated for security reasons. This model is also used by migration to create the database objects.



```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace SafeVaultAPI.Models
4 {
5     1 reference
6     public class RegisterModel
7     {
8         [Required]
9         [StringLength(100)]
10        1 reference
11        public string FullName { get; set; }
12
13        [Required]
14        [EmailAddress]
15        4 references
16        public string Email { get; set; }
17
18        [Required]
19        [Minlength(8)]
20        1 reference
21        public string Password { get; set; }
22    }
23 }
```

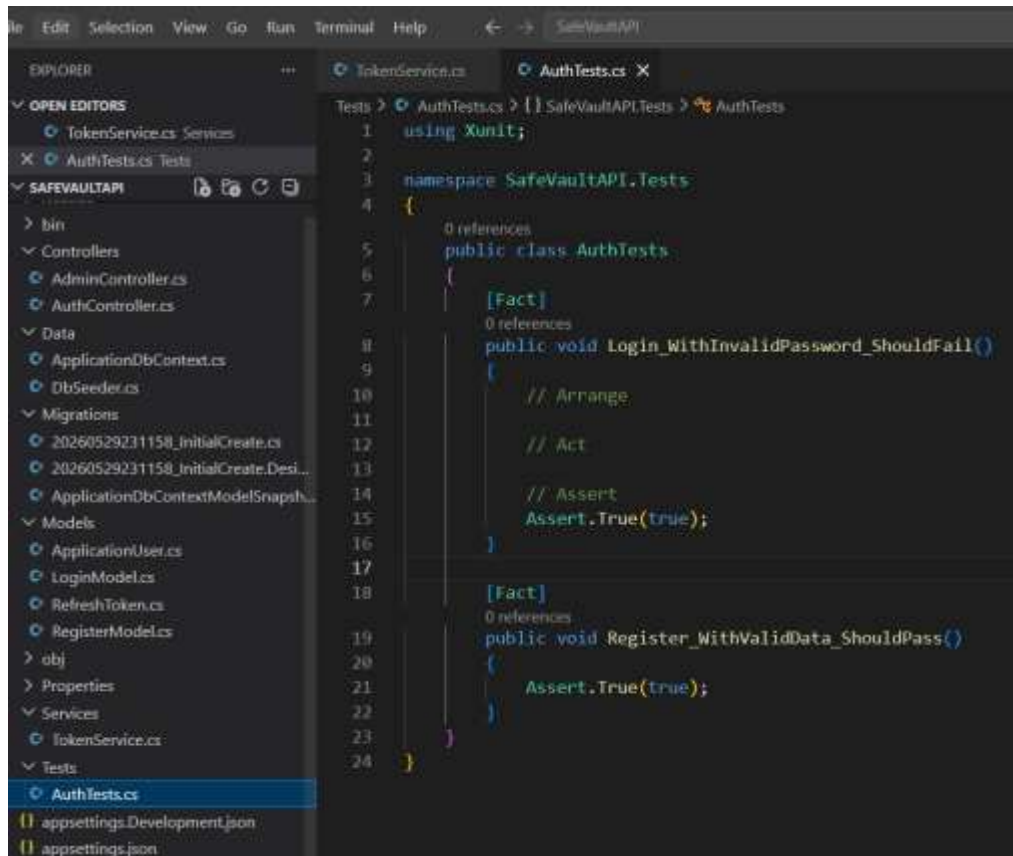
This program defines a very simple TokenService class with one private method:

The function “GenerateRefreshToken()” creates and returns a new GUID (Globally Unique Identifier) as a string. An example is ‘f47ac10b-58cc-4372-a567-0e02b2c3d479’.



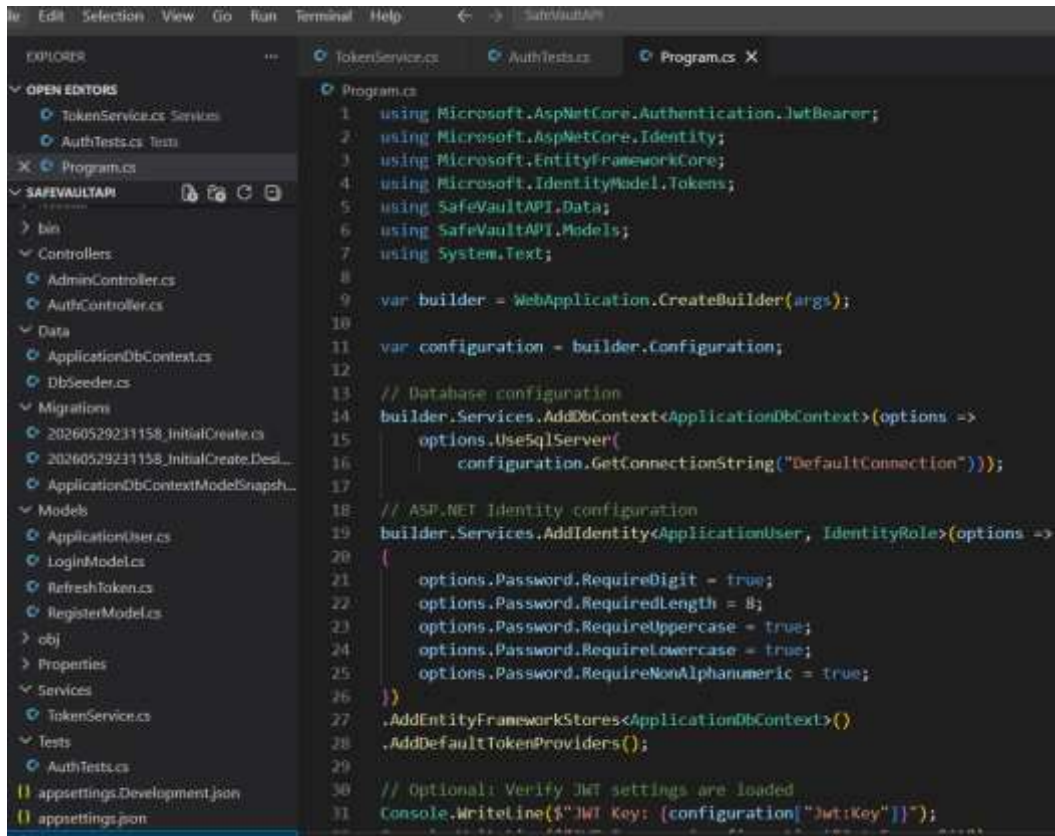
```
1 namespace SafeVaultApi.Services
2 {
3     0 references
4     public class TokenService
5     {
6         0 references
7         private string GenerateRefreshToken()
8         {
9             return Guid.NewGuid().ToString();
10        }
11    }
12 }
```

Kayode Oladipo - assignment

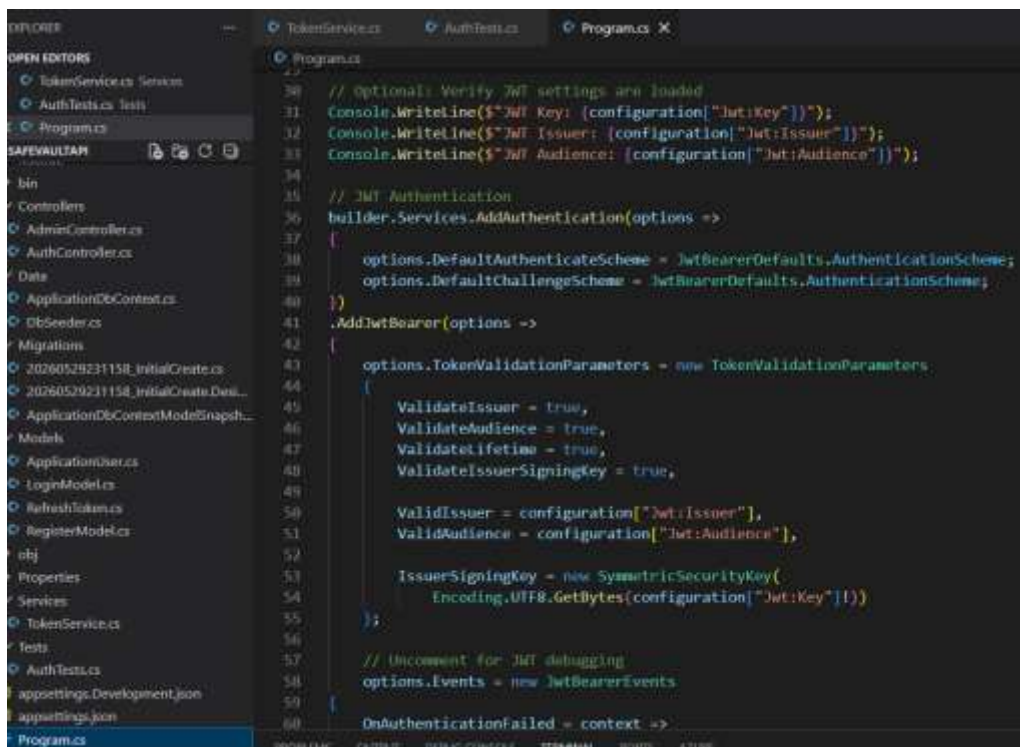


This is where the identity is configured to enforce strong password for security.

Kayode Oladipo - assignment



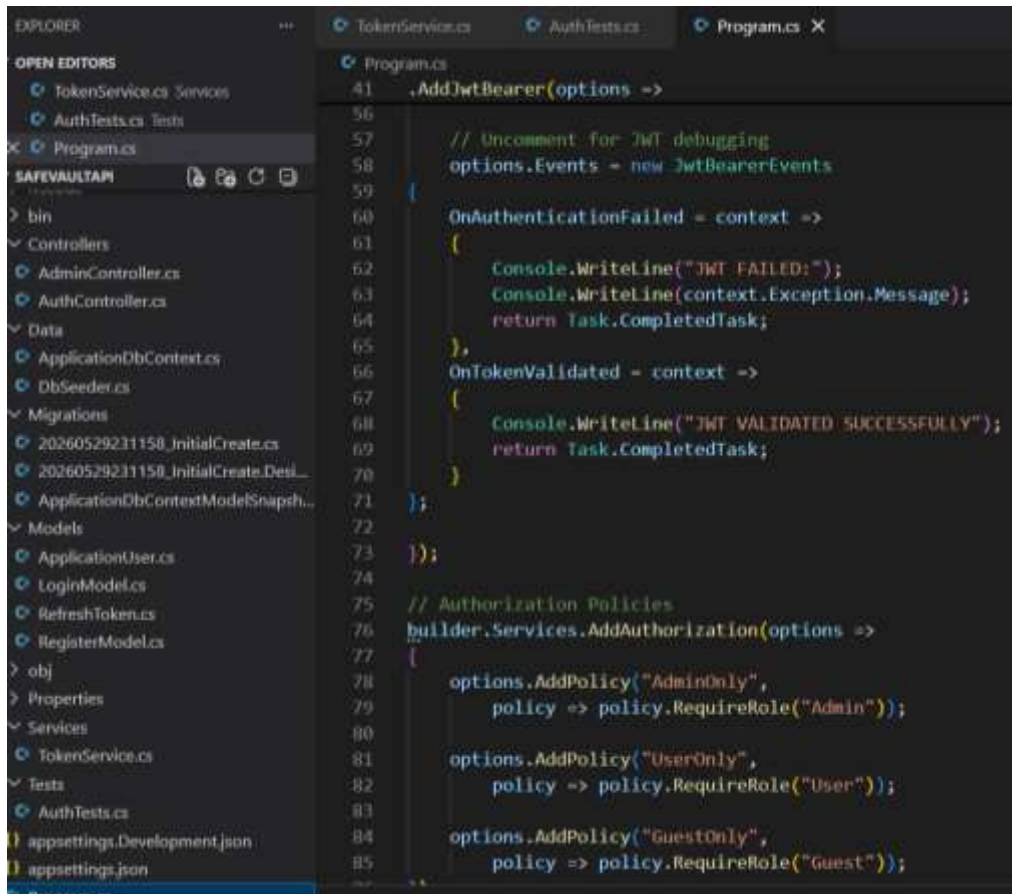
```
1 using Microsoft.AspNetCore.Authentication.JwtBearer;
2 using Microsoft.AspNetCore.Identity;
3 using Microsoft.EntityFrameworkCore;
4 using Microsoft.IdentityModel.Tokens;
5 using SafeVaultAPI.Data;
6 using SafeVaultAPI.Models;
7 using System.Text;
8
9 var builder = WebApplication.CreateBuilder(args);
10
11 var configuration = builder.Configuration;
12
13 // Database configuration
14 builder.Services.AddDbContext<ApplicationDbContext>(options =>
15     options.UseSqlServer(
16         configuration.GetConnectionString("DefaultConnection")));
17
18 // ASP.NET Identity configuration
19 builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
20 {
21     options.Password.RequireDigit = true;
22     options.Password.RequiredLength = 8;
23     options.Password.RequireUppercase = true;
24     options.Password.RequireLowercase = true;
25     options.Password.RequireNonAlphanumeric = true;
26 })
27 .AddEntityFrameworkStores<ApplicationDbContext>()
28 .AddDefaultTokenProviders();
29
30 // Optional: Verify JWT settings are loaded
31 Console.WriteLine($"JWT Key: {configuration["Jwt:Key"]}");
```



```
32 Console.WriteLine($"JWT Issuer: {configuration["Jwt:Issuer"]}");
33 Console.WriteLine($"JWT Audience: {configuration["Jwt:Audience"]}");
34
35 // JWT Authentication
36 builder.Services.AddAuthentication(options =>
37 {
38     options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
39     options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
40 })
41 .AddJwtBearer(options =>
42 {
43     options.TokenValidationParameters = new TokenValidationParameters
44     {
45         ValidateIssuer = true,
46         ValidateAudience = true,
47         ValidateLifetime = true,
48         ValidateIssuerSigningKey = true,
49
50         ValidIssuer = configuration["Jwt:Issuer"],
51         ValidAudience = configuration["Jwt:Audience"],
52
53         IssuerSigningKey = new SymmetricSecurityKey(
54             Encoding.UTF8.GetBytes(configuration["Jwt:Key"]!))
55     };
56
57     // Uncomment for JWT debugging
58     options.Events = new JwtBearerEvents
59     {
60         OnAuthenticationFailed = context =>
```

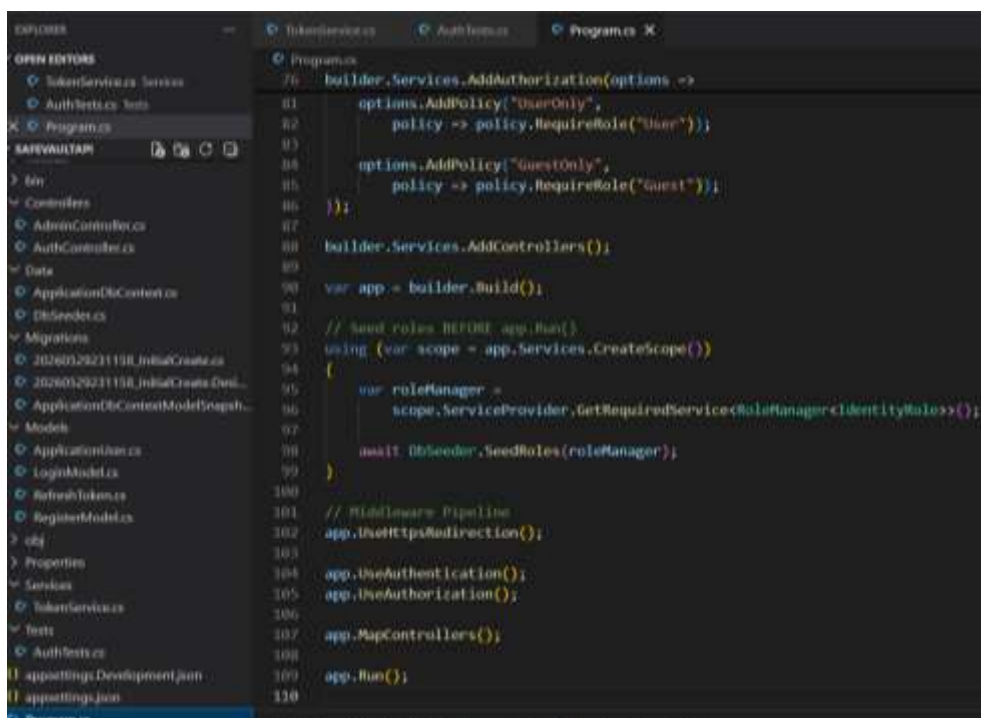
This is where the authorisation policy policies are created.

Kayode Oladipo - assignment



```
41 .AddJwtBearer(options =>
56
57     // Uncomment for JWT debugging
58     options.Events = new JwtBearerEvents
59     {
60         OnAuthenticationFailed = context =>
61         {
62             Console.WriteLine("JWT FAILED:");
63             Console.WriteLine(context.Exception.Message);
64             return Task.CompletedTask;
65         },
66         OnTokenValidated = context =>
67         {
68             Console.WriteLine("JWT VALIDATED SUCCESSFULLY");
69             return Task.CompletedTask;
70         }
71     };
72
73 });
74
75 // Authorization Policies
76 builder.Services.AddAuthorization(options =>
77 {
78     options.AddPolicy("AdminOnly",
79         policy => policy.RequireRole("Admin"));
80
81     options.AddPolicy("UserOnly",
82         policy => policy.RequireRole("User"));
83
84     options.AddPolicy("GuestOnly",
85         policy => policy.RequireRole("Guest"));
```

This is the program where roles are seeded and the pipeline for authentication and authorisation.

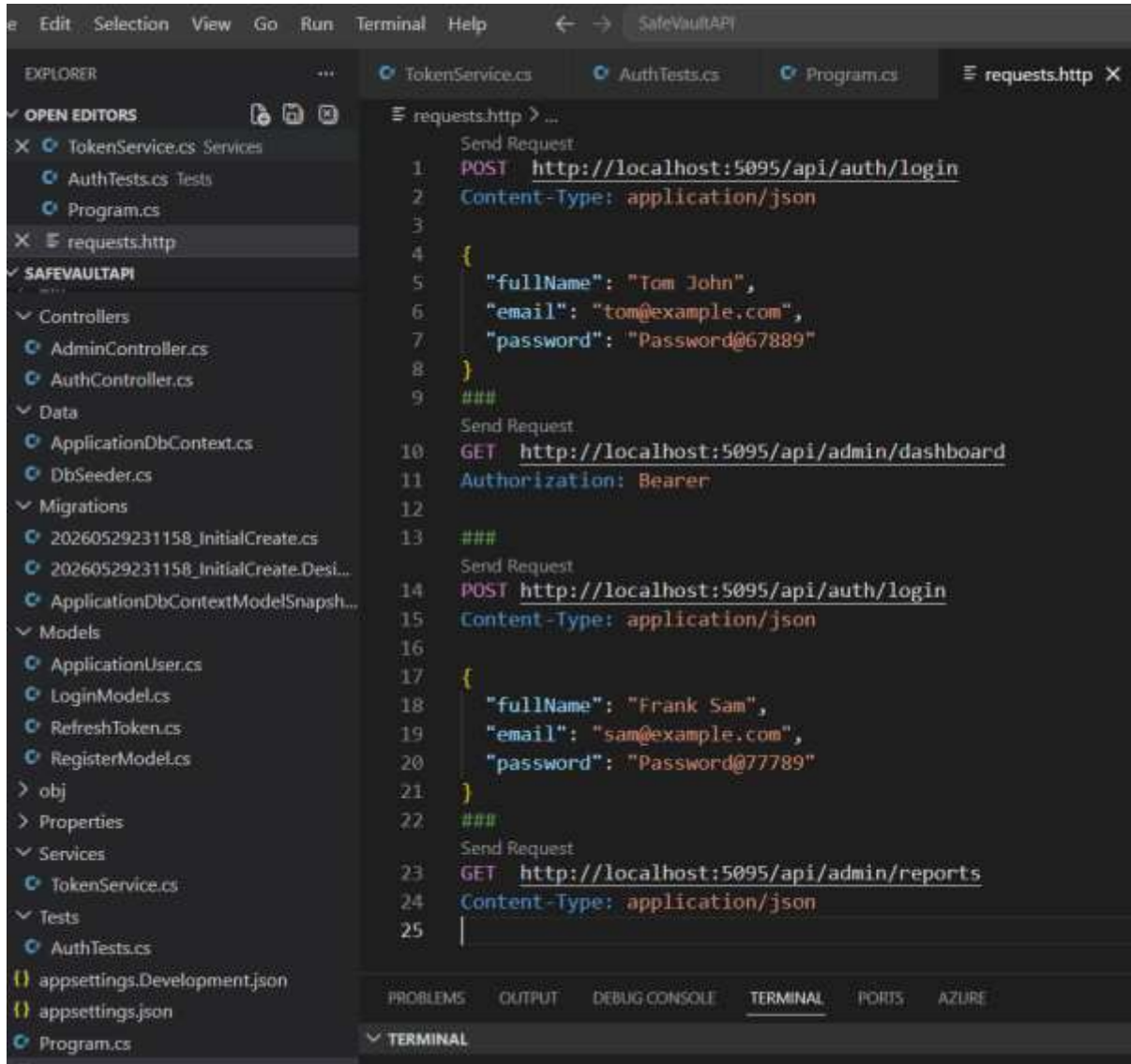


```
76 builder.Services.AddAuthorization(options =>
81     options.AddPolicy("UserOnly",
82         policy => policy.RequireRole("User"));
83
84     options.AddPolicy("GuestOnly",
85         policy => policy.RequireRole("Guest"));
86
87 });
88 builder.Services.AddControllers();
89
90 var app = builder.Build();
91
92 // Seed roles BEFORE app.Run()
93 using (var scope = app.Services.CreateScope())
94 {
95     var roleManager =
96         scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
97
98     await DbSeeder.SeedRoles(roleManager);
99 }
100
101 // Middleware Pipeline
102 app.UseHttpsRedirection();
103
104 app.UseAuthentication();
105 app.UseAuthorization();
106
107 app.MapControllers();
108
109 app.Run();
110
```


TESTING

This is where I tested the application my making API calls .

Kayode Oladipo - assignment



The screenshot shows the Visual Studio Code interface with the following details:

- Explorer Sidebar:** Displays the project structure for 'SAFEVAULTAPI'. It includes folders for 'Controllers', 'Data', 'Migrations', 'Models', 'obj', 'Properties', 'Services', and 'Tests'. Specific files listed include 'AdminController.cs', 'AuthController.cs', 'ApplicationDbContext.cs', 'DbSeeder.cs', 'ApplicationDbContextModelSnapsh...', 'ApplicationUser.cs', 'LoginModel.cs', 'RefreshToken.cs', 'RegisterModel.cs', 'appsettings.Development.json', 'appsettings.json', and 'Program.cs'.
- Open Editors:** Shows 'TokenService.cs', 'AuthTests.cs', 'Program.cs', and the active file 'requests.http'.
- requests.http File Content:**

```
1  POST http://localhost:5095/api/auth/login
2  Content-Type: application/json
3
4  {
5      "fullName": "Tom John",
6      "email": "tom@example.com",
7      "password": "Password@67889"
8  }
9  ###
10 Send Request
11 GET http://localhost:5095/api/admin/dashboard
12 Authorization: Bearer
13 ###
14 Send Request
15 POST http://localhost:5095/api/auth/login
16 Content-Type: application/json
17
18 {
19     "fullName": "Frank Sam",
20     "email": "sam@example.com",
21     "password": "Password@77789"
22 }
23 ###
24 Send Request
25 GET http://localhost:5095/api/admin/reports
26 Content-Type: application/json
```
- Terminal Panel:** Located at the bottom, it shows tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), 'PORTS', and 'AZURE'.